COMP3230 · PRINCIPLES OF OPERATING SYSTEMS

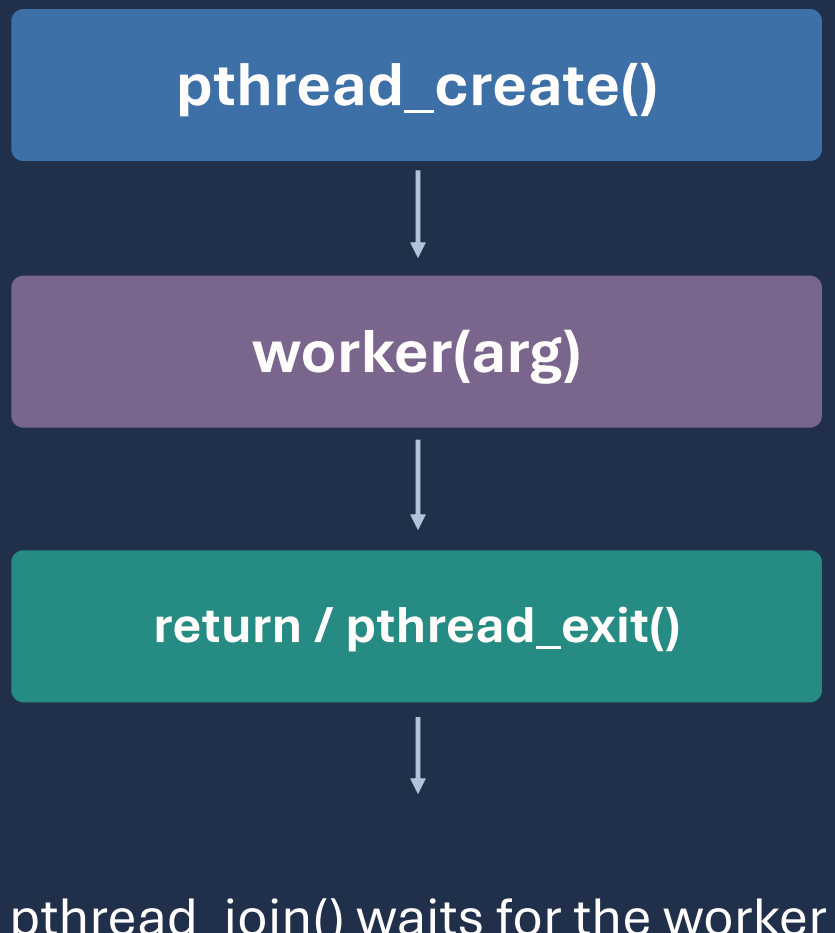
Tutorial 3

Pthread

POSIX threads in C

Create and join threads
Protect shared data
Wait with condition variables

Pei Yuxing yuxing.pei@connect.hku.hk · 2026



What we'll do today

Learn the Pthread calls, then use them in small C programs.

01

Create and manage threads

Start a function, pass data, and collect its result.

create · exit · join · cancel

→

02

Protect shared data

See how updates get lost, then protect them with a mutex.

mutex_init · lock · unlock

→

03

Wait for a condition

Let a thread sleep until the shared state is ready.

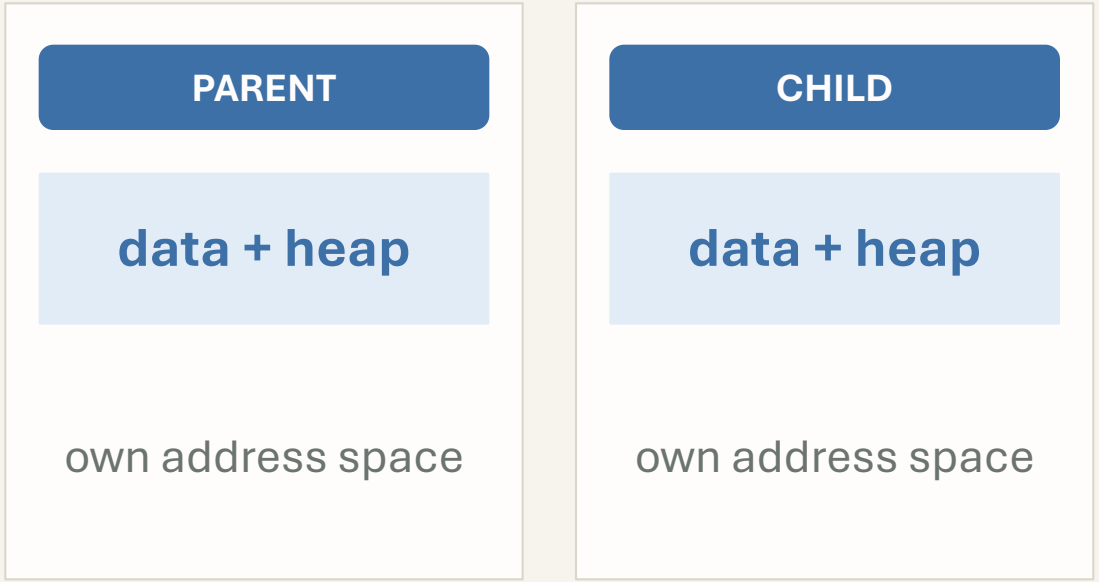
cond_wait · signal · while

Tutorial 1: pointers and structs. Tutorial 2: processes, fork(), and waitpid().

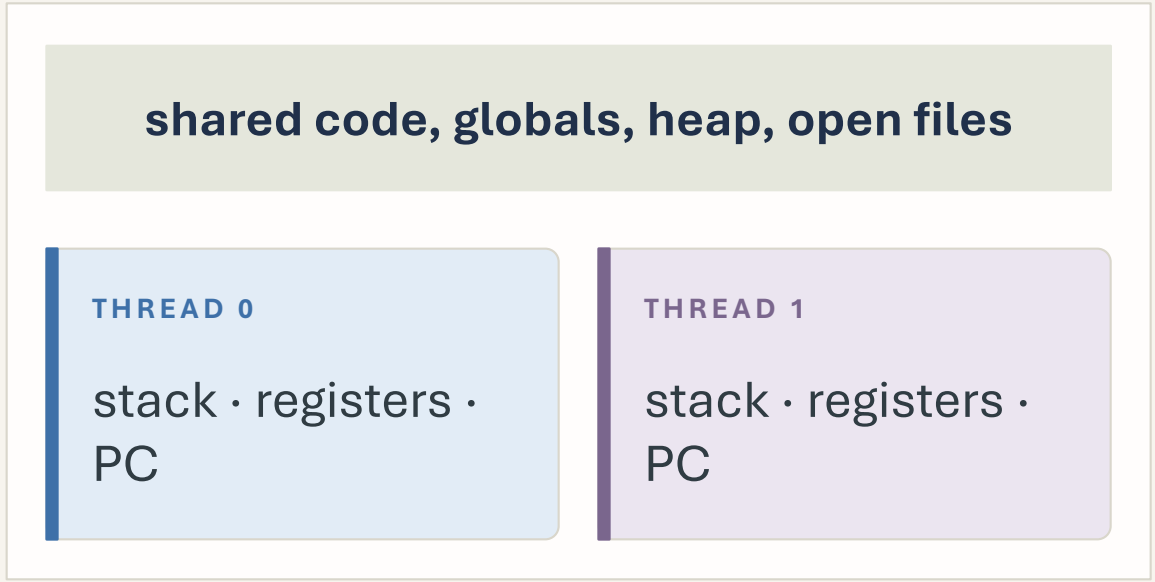
Threads run inside one process

A new thread shares the address space, but has its own stack, registers, and program counter.

TUTORIAL 2: fork()



PTHREAD: pthread_create()



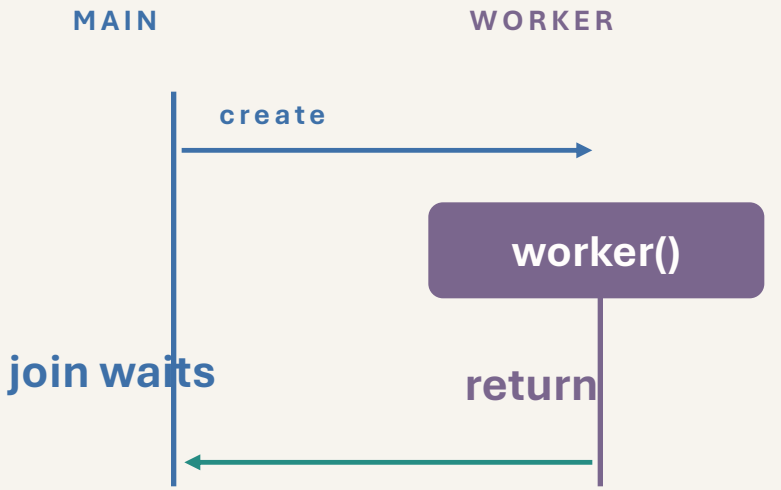
Threads can use pointers to the same objects. Each thread has its own stack and registers.

Your first Pthread program

Pthread is the POSIX thread API. Include pthread.h and compile with -pthread.

hello.c

```c
#include <pthread.h>
#include <stdio.h>
void *worker(void *arg) {
    (void)arg;
    puts("Hello from worker");
    return NULL;
}
int main(void) {
    pthread_t tid;
    int rc = pthread_create(&tid, NULL, worker, NULL);
    if (rc != 0) return 1;
    rc = pthread_join(tid, NULL);
    return rc != 0;
}
```



BUILD + RUN

```
gcc hello.c -pthread -o hello
./hello
```

Check the return value of Pthread calls. Most return 0 on success and an error number on failure.

pthread_create starts a function in a new thread

The worker function takes one void * argument and returns a void *.

CREATE A THREAD

```
pthread_t tid;
int rc = pthread_create(
    &tid,
    NULL,
    worker,
    argument);
```

1. **`&tid`**
   Where the thread ID is stored
2. **`NULL`**
   Use the default attributes
3. **`worker`**
   Function to run: void *worker(void *)
4. **`argument`**
   Pointer passed to worker



> Prepare the argument before pthread_create(). The worker may run before main's next statement.

Pass several arguments in one struct

Double 100 array elements: give each thread half of the array.

STRUCT + START ROUTINE

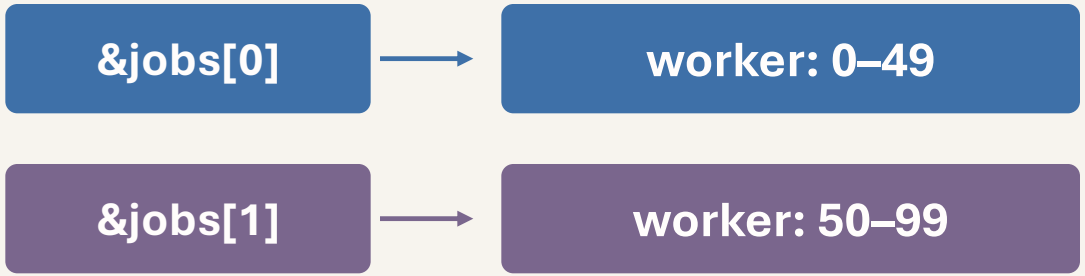
```
typedef struct {
    int *a;
    int begin, end;
    int id;
} Work;

void *worker(void *raw) {
    Work *job = raw;
    for (int i = job->begin;
         i < job->end; i++)
        job->a[i] *= 2;
    return NULL;
}
```

MAIN PREPARES THE INPUTS

ONE Work STRUCT PER THREAD

```
Work jobs[2] = {
    {a,  0,  50, 0},
    {a, 50, 100, 1}
};
```



Keep jobs[] and a[] valid until the workers finish. Each call has its own local loop variable i.

pthread_join waits for a thread to finish

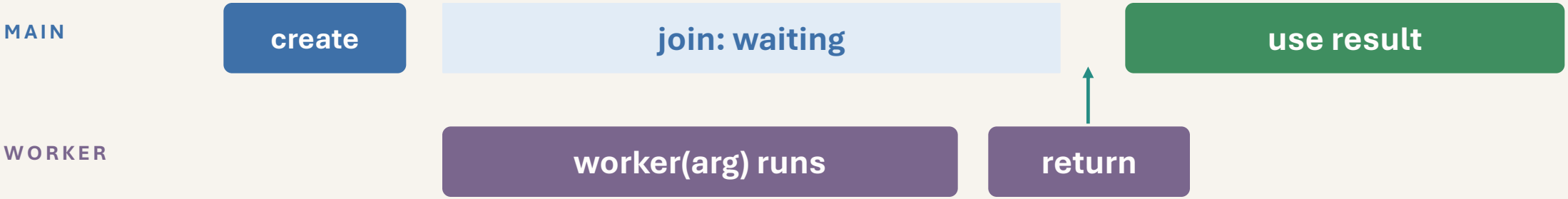
The worker ends with return or pthread_exit(). Main uses join to wait and receive its result pointer.

WORKER: TWO WAYS TO EXIT

```
return result;
// alternatively:
pthread_exit(result);
```

MAIN: WAIT AND RECEIVE THE RESULT

```
void *result;
int rc = pthread_join(tid, &result);
// use result only if rc == 0
```



Pass NULL to pthread_join() when no result pointer is needed.

exit() or returning from main ends the whole process. Returning from worker ends only that thread.

Do not return the address of a local variable

pthread_join receives a pointer; it does not extend the lifetime of the object.

LOCAL VARIABLE: INVALID AFTER RETURN

WORKER: WRONG

```
int id = 7;
return &id;
```

local id → worker returns object no longer exists

Also valid: main provides a result slot and keeps it alive until the join.

HEAP ALLOCATION: VALID UNTIL free()

WORKER: RETURN A RESULT

```
int *p = malloc(sizeof *p);
if (p == NULL) return NULL;
*p = 7;
return p;
```

MAIN: RECEIVE AND FREE THE RESULT

```
void *p;
if (pthread_join(tid, &p) == 0 && p) {
    use(*(int *)p); free(p);
}
```

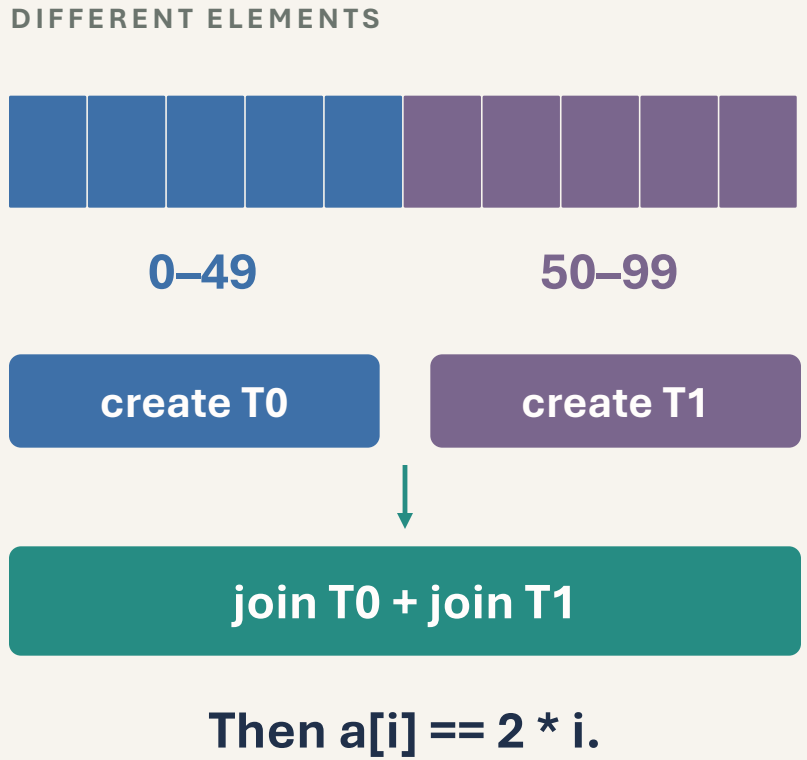
Use <stdlib.h> for malloc/free. In this example, worker allocates the result and main frees it.

Create both threads, then join both

Use Work and worker() from page 6. Each thread doubles its assigned half of the array.

MAIN: ERROR CHECKS OMITTED

```
int a[100];
for (int i = 0; i < 100; i++) a[i] = i;
Work jobs[2] = {
    {a, 0, 50, 0}, {a, 50, 100, 1}
};
pthread_t threads[2];
for (int i = 0; i < 2; i++)
    pthread_create(&threads[i], NULL,
                   worker, &jobs[i]);
for (int i = 0; i < 2; i++)
    pthread_join(threads[i], NULL);
use_array(a);
```



Create both threads before joining either one. Check each Pthread return code in a complete program.

Use pthread_t to identify a thread

pthread_create writes the new thread's ID. pthread_self() returns the calling thread's ID.

ONE PROCESS: getpid() same PID for every thread

Thread 0	`pthread_self() → pthread_t`	`gettid() → Linux TID`
Thread 1	`pthread_self() → pthread_t`	`gettid() → Linux TID`

COMPARE PTHREAD IDs

```
int same = pthread_equal(tid, pthread_self());
// nonzero means: the same thread
```

YOUR OWN WORKER NUMBER

job->id is just 0 or 1.
It is separate from pthread_t.

Use pthread_equal() to compare thread IDs. gettid() returns a separate Linux-specific ID.

pthread_cancel requests another thread to stop

By default, the request is handled at a cancellation point, such as pthread_testcancel().

DEFAULT: CANCELLATION ENABLED, TYPE DEFERRED

REQUEST

pthread_cancel(tid)

→

EXAMPLE CANCELLATION POINT

pthread_testcancel()

→

WAIT FOR EXIT

pthread_join(tid, ...)

REQUEST + WAIT: CHECK RETURN CODES

```
pthread_cancel(tid);
void *result;
pthread_join(tid, &result);
// cancelled: result == PTHREAD_CANCELED
```

BEFORE THE THREAD EXITS

Arrange cleanup for any locks or resources the worker holds.

Cancellation is a request, not an immediate kill. Extra settings and cleanup are on reference page 26.

From process calls to Pthread calls

The lifecycle is familiar from Tutorial 2. Sharing an address space changes how threads exchange data.

TASK	PROCESS	PTHREAD
Create	fork()	pthread_create()
Identify	getpid()	pthread_self()
End itself	exit()	return / pthread_exit()
Wait + collect	waitpid()	pthread_join()
Exchange data	IPC or explicit shared memory	Pointers to shared objects

The threads can now run and finish. Next: what if both of them write to the same variable?

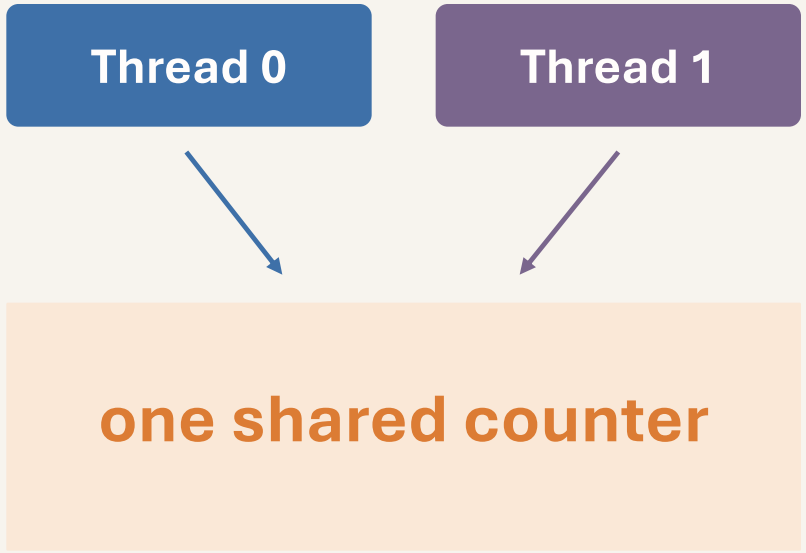
Two threads can lose updates to a shared counter

Each thread adds 1 a million times. The intended total is 2000000, but counter++ can lose updates.

BOTH THREADS RUN THIS FUNCTION

```
int counter = 0;

void *count(void *arg) {
    (void)arg;
    for (int i = 0; i < 1000000; i++)
        counter++;
    return NULL;
}
```



Different array elements were safe.
This time both threads change one int.

> Unsynchronized conflicting accesses cause a C data race. The behavior is undefined.

counter++ reads, adds 1, and writes back

T0 and T1 both read 41, then both write 42. Only one increment remains.

STEP	THREAD 0: LOCAL r0	THREAD 1: LOCAL r1	counter
1	read 41 into r0	—	41
2	—	read 41 into r1	41
3	write r0 + 1	—	42
4	—	write r1 + 1	42

Expected: 43

Illustrated result: 42

The table shows one possible failure. Next, use a Pthread mutex to protect the whole update.

Protect counter++ with a Pthread mutex

Only one thread can hold lock at a time. The other thread waits in pthread_mutex_lock().

MUTEX LIFETIME + PROTECTED UPDATE

```
pthread_mutex_t lock;

// before starting the threads
pthread_mutex_init(&lock, NULL);

// in each worker's loop
pthread_mutex_lock(&lock);
counter++;
pthread_mutex_unlock(&lock);

// after all threads have joined
pthread_mutex_destroy(&lock);
```

1 **INITIALIZE**

Before starting the workers.

2 **LOCK → UPDATE → UNLOCK**

Lock before updating counter.

3 **DESTROY**

After both workers have joined.

After both joins: counter == 2000000

Static alternative: pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER; use it instead of init().

Update processed and remaining under one lock

For the 100-element array, processed counts finished elements; remaining counts those still to do.

Initial values: processed = 0, remaining = 100

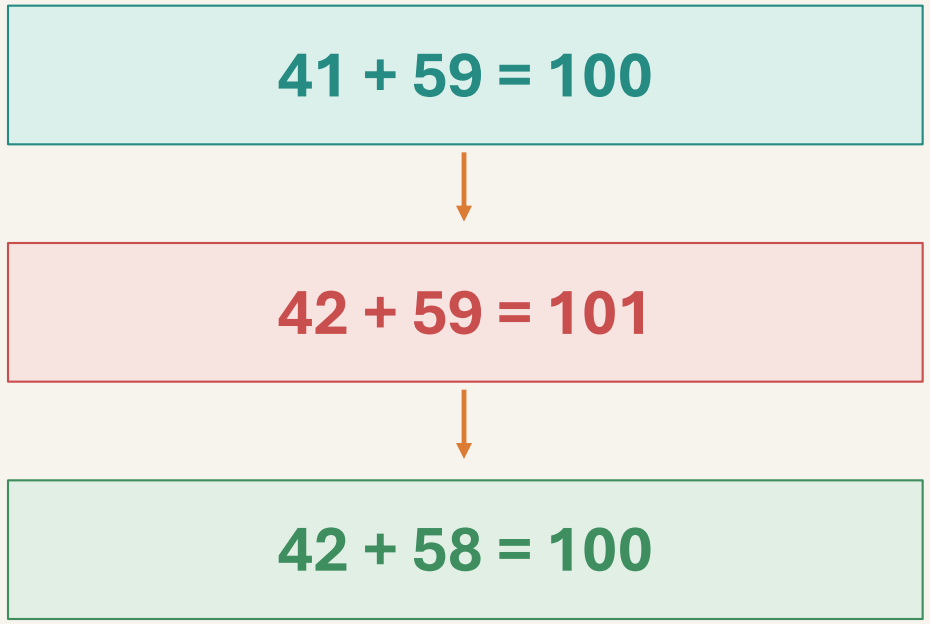
WORKER: UPDATE THE PAIR

```
pthread_mutex_lock(&lock);
processed++;
remaining--;
pthread_mutex_unlock(&lock);
```

READER: COPY THE PAIR UNDER THE SAME LOCK

```
pthread_mutex_lock(&lock);
int p = processed, r = remaining;
pthread_mutex_unlock(&lock);
show_progress(p, r);
```

processed + remaining



Readers must see a sum of 100.

Keep both updates under one lock. A reader takes the same lock to read a consistent pair.

pthread_mutex_trylock returns immediately

If the lock for shared data A is busy, do independent work B instead.

```c
#include <errno.h>

int rc = pthread_mutex_trylock(&lock);
if (rc == 0) {
    update_a();
    pthread_mutex_unlock(&lock);
} else if (rc == EBUSY) {
    update_b(); // independent work
} else {
    handle_error(rc);
}
```

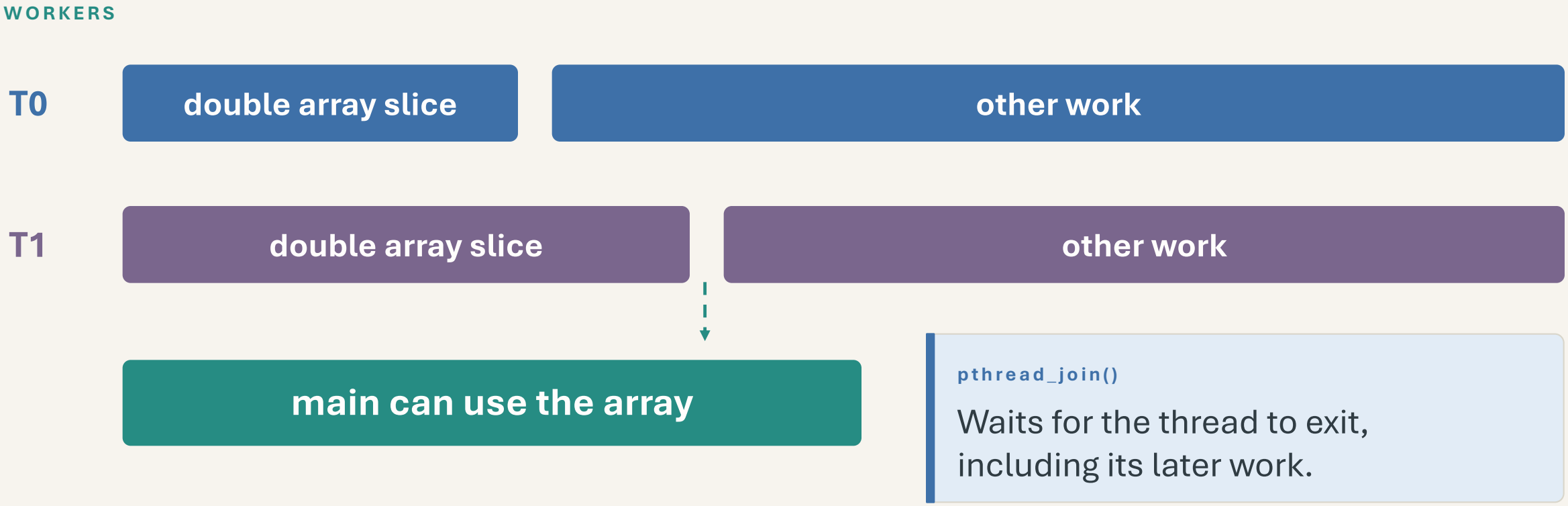
RETURN CODE

`0`	Got the lock: update A.
`EBUSY`	Lock is busy: work on B.
`other`	Handle the error.

> Getting the lock allows access to the data. It does not mean the result you need is ready.

Use the array before the workers exit

Main needs both slices to be ready. The workers still have other work to do afterward.



Main needs to wait for the array. The workers' later work does not read or write it.

Use done flags, a mutex, and a condition variable

Main waits until done[0] and done[1] are both 1: both array slices are ready.

TWO COMPLETION FLAGS

done[0], done[1]

0 = still working; 1 = slice ready.

MUTEX

lock

Hold lock to read or change done.

CONDITION VARIABLE

cond

Main waits; workers signal.

INITIALIZE BEFORE STARTING WORKERS

```
int done[2] = {0, 0};
pthread_mutex_t lock;
pthread_cond_t cond;
pthread_mutex_init(&lock, NULL);
pthread_cond_init(&cond, NULL);
```

DESTROY AFTER ALL USERS HAVE FINISHED

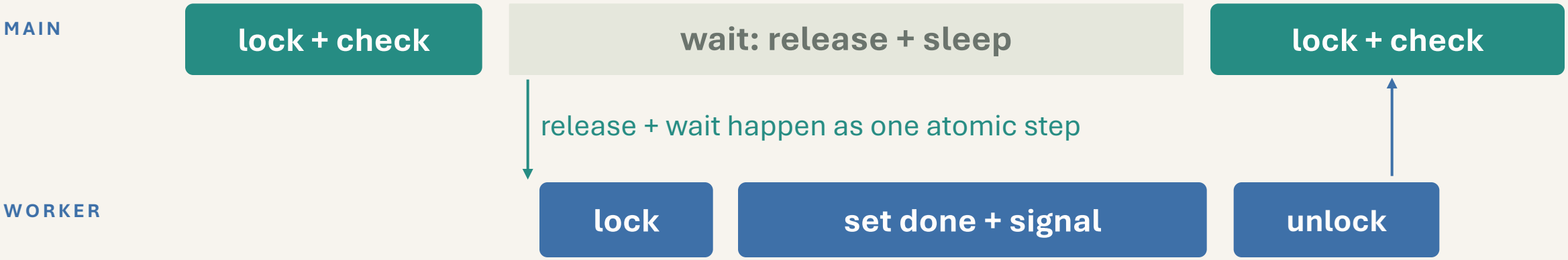
```
// after joining the workers
pthread_cond_destroy(&cond);
pthread_mutex_destroy(&lock);
```

Static alternative: PTHREAD_COND_INITIALIZER

signal wakes at least one current waiter, if any. broadcast wakes all. Main still checks done.

pthread_cond_wait lets workers acquire the mutex

Main holds lock to check done. cond_wait releases lock while waiting, then locks it again before returning.



MAIN: CHECK THE CONDITION AGAIN

```
pthread_mutex_lock(&lock);
while (!done[0] || !done[1])
    pthread_cond_wait(&cond, &lock);
pthread_mutex_unlock(&lock);
```

WHEN WAIT RETURNS

Main has lock again.
It checks both done flags.

A separate unlock followed by sleep can miss a signal. pthread_cond_wait handles that transition.

Set done, then wake main with pthread_cond_signal

Each worker finishes its slice, sets done, and signals. Main waits until both slices are ready.

WORKER: FINISH ITS SLICE, THEN SIGNAL

```
void *worker(void *raw) {
    Work *job = raw;
    double_slice(job);
    pthread_mutex_lock(&lock);
    done[job->id] = 1;
    pthread_cond_signal(&cond);
    pthread_mutex_unlock(&lock);
    other_work(job->id);
    return NULL;
}
```

MAIN: WAIT FOR BOTH done FLAGS

```
pthread_mutex_lock(&lock);
while (!done[0] || !done[1])
    pthread_cond_wait(&cond, &lock);
pthread_mutex_unlock(&lock);

use_array(a);

for (int i = 0; i < 2; i++)
    pthread_join(threads[i], NULL);
```



Only main waits on cond here. The workers continue with other_work(), which leaves the array alone.

Use while: a wakeup does not mean both slices are ready

Check done before waiting and after each wakeup. Continue only when both flags are 1.

BOTH SLICES READY BEFORE MAIN WAITS

done is already [1,1].
Main checks the flags and skips wait.

ONLY T0'S SLICE READY WHEN MAIN WAKES

done is [1,0].
Main waits for T1's slice to be ready.



HOLD THE MUTEX WHILE CHECKING

```
while (!done[0] || !done[1])
    pthread_cond_wait(&cond, &lock);
```

A wait can also return without a signal (a spurious wakeup). Use while, not if.

Signals are not saved for later. The done flags still record which slices are ready.

Choose the Pthread call for the task

Run and wait for threads, protect shared updates, or wait until a condition is true.

START, END, AND WAIT

`pthread_create()`

Start worker(argument).

`return / pthread_exit()`

End the calling thread.

`pthread_join()`

Wait for exit; receive the result pointer.

PROTECT SHARED DATA

`pthread_mutex_init()`

Initialize before use.

`pthread_mutex_lock/unlock()`

Protect shared reads and writes.

`pthread_mutex_destroy()`

Destroy after all users finish.

WAIT UNTIL DATA IS READY

`pthread_cond_init()`

Initialize before any wait or signal.

`pthread_cond_wait()`

Wait in while; check done again.

`pthread_cond_signal/broadcast()`

Wake waiting threads to check again.

Build with -pthread, check return codes, and keep arguments and shared objects alive until their users finish.

C11 atomics are another option for one counter

atomic_fetch_add belongs to <stdatomic.h>. It is separate from the Pthread API.

C11 ATOMIC COUNTER

```c
#include <stdatomic.h>

_Atomic int counter = 0;

// each worker performs an atomic increment
atomic_fetch_add(&counter, 1);
```



BOTH UPDATES ARE COUNTED

With 1000000 increments per worker: counter == 2000000 after both joins.

ATOMIC OPERATION	PTHREAD MUTEX
One operation on an atomic object	Several statements under one lock

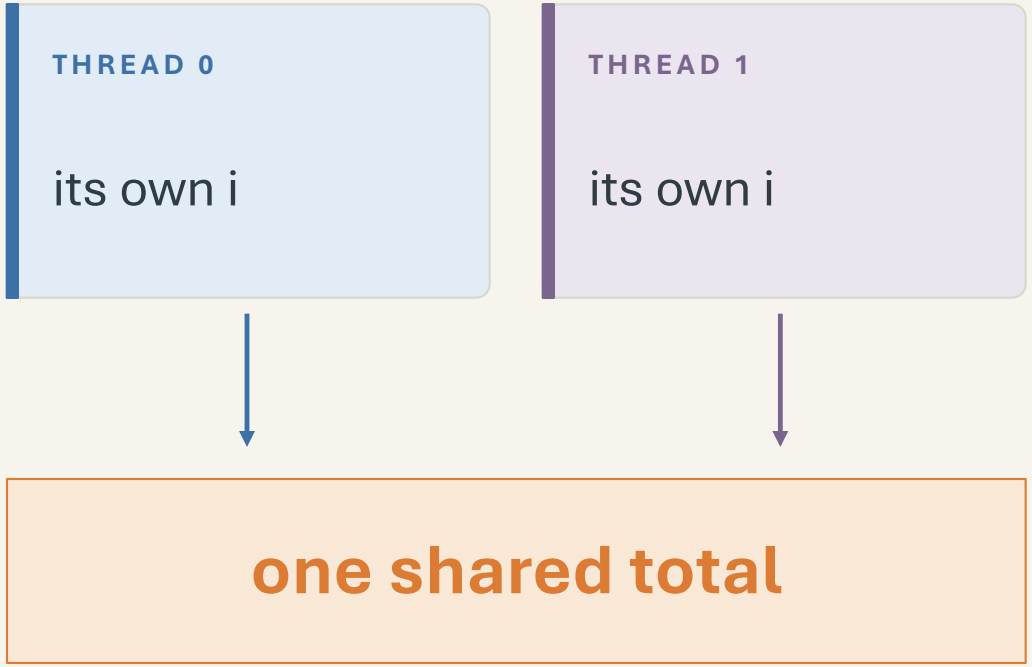
Two separate atomic updates do not make a pair atomic. Atomic operations are not always lock-free.

Local i is separate; static total is shared

Each call has its own int i. All calls use the same static int total.

INSIDE worker()

```
void *worker(void *arg) {
    int i = 0;
    static int total = 0;
    // ...
    return NULL;
}
```



Declaring a variable inside a function does not always make it private to a thread.

Control when cancellation can take effect

Default: cancellation is enabled and handled at a cancellation point.

CHANGE CANCELLATION SETTINGS

```
pthread_setcancelstate(
    PTHREAD_CANCEL_DISABLE, NULL);
// short region where cancellation is unsafe
pthread_setcancelstate(
    PTHREAD_CANCEL_ENABLE, NULL);

pthread_setcanceltype(
    PTHREAD_CANCEL_DEFERRED, NULL);
```

DEFERRED

At the next cancellation point, e.g. pthread_testcancel().

ASYNCHRONOUS

Can interrupt almost any operation. Cleanup is difficult.

Install cleanup handlers with pthread_cleanup_push/pop when cancellation could leave a mutex locked or a resource held.

> A cancelled joinable thread still needs pthread_join(). A request made while disabled remains pending.